EXPERIMENT 1

Write the Python program to solve the 8-Puzzle problem

Register Number: **192511013**

192511013

Aim

To write a Python program that solves the 8-Puzzle problem by moving the blank tile through the 3x3 board, from a given initial configuration to the goal configuration, using the A* search algorithm with the Manhattan-distance heuristic.

Algorithm

Step 1: Represent each board state as a tuple of 9 numbers, where 0 is the blank tile. Fix the goal state.

Step 2: Define the heuristic h(n) as the sum of the Manhattan distances of every tile from its goal position (blank excluded).

Step 3: Create a priority queue (min-heap) ordered by f(n) = g(n) + h(n), where g(n) is the number of moves made so far. Insert the initial state with g = 0.

Step 4: Remove the state with the smallest f(n) from the queue. If it is already visited, skip it; otherwise mark it visited and record its parent.

Step 5: If the removed state equals the goal state, trace the parent links back to the start to get the solution path, then go to step 7.

Step 6: Generate all neighbours by sliding the blank up, down, left and right (where legal). For each neighbour with a better g value, push it into the queue with its new f value. Repeat from step 4.

Step 7: Print the number of moves and every board configuration along the path. If the queue becomes empty, print that no solution exists.

192511013

Program

```
import heapq

GOAL = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)

def manhattan(state):
    dist = 0
    for i, tile in enumerate(state):
        if tile != 0:
            g = GOAL.index(tile)
            dist += abs(i // 3 - g // 3) + abs(i % 3 - g % 3)
    return dist

def neighbours(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            n = nr * 3 + nc
            s = list(state)
            s[z], s[n] = s[n], s[z]
            yield tuple(s)

def solve(start):
    pq = [(manhattan(start), 0, start, None)]
    parent = {}
    best_g = {start: 0}
    while pq:
        f, g, state, prev = heapq.heappop(pq)
        if state in parent:
            continue
        parent[state] = prev
        if state == GOAL:
            path = []
            while state is not None:
                path.append(state)
                state = parent[state]
            return path[::-1]
        for nxt in neighbours(state):
            ng = g + 1
            if nxt not in best_g or ng < best_g[nxt]:
                best_g[nxt] = ng
                heapq.heappush(pq, (ng + manhattan(nxt), ng, nxt, state))
    return None

def show(state):
    for i in range(0, 9, 3):
        print(' '.join(str(x) if x else '_' for x in state[i:i + 3]))
    print()

if __name__ == "__main__":
    start = (2, 8, 3,
             1, 6, 4,
             7, 0, 5)
    path = solve(start)
    if path:
        print("Solved in", len(path) - 1, "moves\n")
        for step, s in enumerate(path):
            print("Step", step)
            show(s)
    else:
        print("No solution exists")
```

192511013

Output

```
Solved in 5 moves

Step 0
2 8 3
1 6 4
7 _ 5

Step 1
2 8 3
1 _ 4
7 6 5

Step 2
2 _ 3
1 8 4
7 6 5

Step 3
_ 2 3
1 8 4
7 6 5

Step 4
1 2 3
_ 8 4
7 6 5

Step 5
1 2 3
8 _ 4
7 6 5
```

192511013